

**MY LEETCODE**

# **INTERVIEW NOTES**

42 problems | Questions | Examples | Complexity | Solutions

Focused notes for fast revision.

# 0001. 1. Two Sum

Difficulty: Easy Language: C++

## QUESTION

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

## EXAMPLE

Input: `nums = [2,7,11,15]`, `target = 9` Output: `[0,1]` Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

## COMPLEXITY

Time:  $O(n)$  average Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> mp;
        for (int i = 0; i < nums.size(); i++) {
            int x = target - nums[i];
            if (mp.find(x) != mp.end()) {
                return {mp[x], i};
            }
            mp[nums[i]] = i;
        }
        return {};
    }
};
```

# 0002. 2. Add Two Numbers

Difficulty: Medium Language: C++

## QUESTION

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

## EXAMPLE

Input: l1 = [2,4,3], l2 = [5,6,4] Output: [7,0,8] Explanation: 342 + 465 = 807.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *   int val;
 *   ListNode *next;
 *   ListNode() : val(0), next(nullptr) {}
 *   ListNode(int x) : val(x), next(nullptr) {}
 *   ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummy = new ListNode(0);
        ListNode* temp = dummy;
        int carry = 0;
        while (l1 != NULL || l2 != NULL || carry != 0) {
            int sum = carry;
            if (l1 != NULL) {
                sum += l1->val;
                l1 = l1->next;
            }
            if (l2 != NULL) {
                sum += l2->val;
            }
            int digit = sum % 10;
            carry = sum / 10;
            temp->val = digit;
            if (temp->next == NULL) {
                temp->next = new ListNode(0);
            }
            temp = temp->next;
        }
        if (carry != 0) {
            temp->val = carry;
        }
        return dummy;
    }
};
```

```
l2 = l2->next;
}
carry = sum / 10;
temp->next = new ListNode(sum % 10);
temp = temp->next;
}
return dummy->next;
}
};
```

# 0003. 3. Longest Substring Without Repeating Characters

Difficulty: Medium Language: C++

## QUESTION

Given a string *s*, find the length of the longest substring without duplicate characters.

## EXAMPLE

Input: *s* = "abcabcbb" Output: 3 Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

## COMPLEXITY

Time:  $O(n)$  average Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> st;
        int left = 0;
        int maxi = 0;
        for (int right = 0; right < s.size(); right++) {
            while(st.find(s[right]) != st.end()) {
                st.erase(s[left]);
                left++;
            }
            st.insert(s[right]);
            maxi = max(maxi, right - left + 1);
        }
        return maxi;
    }
};
```

# 0004. 4. Median of Two Sorted Arrays

Difficulty: Hard Language: C++

## QUESTION

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

## EXAMPLE

Input: `nums1 = [1,3]`, `nums2 = [2]` Output: `2.00000` Explanation: merged array = `[1,2,3]` and median is `2`.

## COMPLEXITY

Time:  $O(n \log n)$  Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        nums1.insert(nums1.end(), nums2.begin(), nums2.end());
        sort(nums1.begin(), nums1.end());
        if(nums1.size()%2==0){
            return (nums1[(nums1.size()/2)-1]+nums1[nums1.size()/2])/2.0;
        }
        else{
            return nums1[nums1.size()/2.0];
        }
        return{};
    }
};
```

# 0007. 7. Reverse Integer

Difficulty: Medium Language: C++

## QUESTION

Given a signed 32-bit integer  $x$ , return  $x$  with its digits reversed. If reversing  $x$  causes the value to go outside the signed 32-bit integer range  $[-2^{31}, 2^{31} - 1]$ , then return 0.

## EXAMPLE

Input:  $x = 123$  Output: 321

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int reverse(int x) {
        int rev = 0;
        while(x!=0){
            int d = x%10;
            if(rev > INT_MAX/10 || rev < INT_MIN/10)
                return 0;
            rev = rev*10 + d;
            x=x/10;
        }
        return rev;
    }
};
```

# 0009. 9. Palindrome Number

Difficulty: Easy Language: C++

## QUESTION

Given an integer x, return true if x is a palindrome, and false otherwise.

## EXAMPLE

Input: x = 121 Output: true Explanation: 121 reads as 121 from left to right and from right to left.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
class Solution {
public:
    bool isPalindrome(int x){
        if (x < 0)
            return false;
        if (x != 0 && x % 10 == 0)
            return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x /= 10;
        }
        if(x==rev || x==rev/10)
            return true;
        else
            return false;
    }
};
```



# 0011. 11. Container With Most Water

Difficulty: Medium Language: C++

## QUESTION

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are `(i, 0)` and `(i, height[i])`.

## EXAMPLE

Input: `height = [1,8,6,2,5,4,8,3,7]` Output: 49 Explanation: The above vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size()-1;
        int area = 0;
        while(left<right){
            int h = min(height[left],height[right]);
            int a = h*(right-left);
            area = max(a,area);
            if(height[left]<height[right]){
                left++;
            }
            else{
                right--;
            }
        }
        return area;
    }
};
```

# 0014. 14. Longest Common Prefix

Difficulty: Easy Language: C++

## QUESTION

Write a function to find the longest common prefix string amongst an array of strings.

## EXAMPLE

Input: strs = ["flower","flow","flight"] Output: "fl"

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        string prefix = strs[0];
        for(int i = 1; i < strs.size(); i++) {
            int j = 0;
            while(j < prefix.size() &&
                j < strs[i].size() &&
                prefix[j] == strs[i][j]) {
                j++;
            }
            prefix = prefix.substr(0, j);
            if(prefix == "")
                return "";
        }
        return prefix;
    }
};
```

# 0019. 19. Remove Nth Node From End of List

Difficulty: Medium Language: C++

## QUESTION

Given the head of a linked list, remove the nth node from the end of the list and return its head.

## EXAMPLE

Input: head = [1,2,3,4,5], n = 2 Output: [1,2,3,5]

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *   int val;
 *   ListNode *next;
 *   ListNode() : val(0), next(nullptr) {}
 *   ListNode(int x) : val(x), next(nullptr) {}
 *   ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *dummy = new ListNode(0);
        dummy->next = head;
        ListNode *temp=head;
        int count = 0;
        while(temp!=NULL){
            temp = temp->next;
            count++;
        }
        int k=count-n;
        temp = dummy;
```

```
while(k>0){
temp=temp->next;
k--;
}
ListNode *templ=temp->next;
temp->next=templ->next;
delete templ;
ListNode* newHead = dummy->next;
delete dummy;
return newHead;
}
};
```

# 0021. 21. Merge Two Sorted Lists

Difficulty: Easy Language: C++

## QUESTION

You are given the heads of two sorted linked lists list1 and list2.

## EXAMPLE

Input: list1 = [1,2,4], list2 = [1,3,4] Output: [1,1,2,3,4,4]

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *   int val;
 *   ListNode *next;
 *   ListNode() : val(0), next(nullptr) {}
 *   ListNode(int x) : val(x), next(nullptr) {}
 *   ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL){return list2;}
        if(list2==NULL){return list1;}
        ListNode* dummy=new ListNode(-1);
        ListNode* temp = dummy;
        while(list1!=NULL && list2!=NULL){
            if(list1->val<=list2->val){
                temp->next=list1;
                list1=list1->next;
            }
            else{
                temp->next=list2;
                list2=list2->next;
            }
        }
    }
};
```

```
}  
temp=temp->next;  
}  
if(list1!=NULL){  
temp->next=list1;  
}  
else{  
temp->next=list2;  
}  
return dummy->next;  
}  
};
```

# 0024. 24. Swap Nodes in Pairs

Difficulty: Medium Language: C++

## QUESTION

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

## EXAMPLE

No example extracted from the README.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *   int val;
 *   ListNode *next;
 *   ListNode() : val(0), next(nullptr) {}
 *   ListNode(int x) : val(x), next(nullptr) {}
 *   ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* temp = dummy;
        while(temp->next != NULL && temp->next->next != NULL) {
            ListNode* first = temp->next;
            ListNode* second = first->next;
            first->next = second->next;
            second->next = first;
            temp->next = second;
            temp = first;
        }
    }
};
```

```
ListNode* newHead = dummy->next;  
delete dummy;  
return newHead;  
}  
};
```



# 0033. 33. Search in Rotated Sorted Array

Difficulty: Medium Language: C++

## QUESTION

There is an integer array `nums` sorted in ascending order (with distinct values).

## EXAMPLE

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0` Output: 4

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        for(int i=0;i<nums.size();i++){
            if(nums[i]==target){
                return i;
            }
        }
        return -1;
    }
};
```

# 0034. 34. Find First and Last Position of Element in Sorted Array

Difficulty: Medium Language: C++

## QUESTION

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

## EXAMPLE

Input: `nums = [5,7,7,8,8,10]`, `target = 8` Output: `[3,4]`

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = -1;
        int last = -1;
        // Find first occurrence
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                first = mid;
                right = mid - 1; // search left
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        // Find last occurrence
        left = 0;
```

```
right = nums.size() - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (nums[mid] == target) {
        last = mid;
        left = mid + 1; // search right
    }
    else if (target > nums[mid]) {
        left = mid + 1;
    }
    else {
        right = mid - 1;
    }
}
return {first, last};
};
```

# 0035. 35. Search Insert Position

Difficulty: Easy Language: C++

## QUESTION

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

## EXAMPLE

Input: nums = [1,3,5,6], target = 5 Output: 2

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size()-1;
        while(left<=right){
            int mid = left + (right - left)/2;
            if(nums[mid]==target){
                return mid;
            }
            if(target>nums[mid]){
                left=mid+1;
            }
            else{
                right=mid-1;
            }
        }
        return left;
    }
};
```

# 0041. 41. First Missing Positive

Difficulty: Hard Language: C++

## QUESTION

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

## EXAMPLE

Input: `nums = [1,2,0]` Output: 3 Explanation: The numbers in the range `[1,2]` are all in the array.

## COMPLEXITY

Time:  $O(n)$  average Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        unordered_set<int> st;
        for(int x:nums){
            st.insert(x);
        }
        for(int i =1;i<=nums.size();i++){
            if(st.find(i)==st.end()){
                return i;
            }
        }
        return nums.size()+1;
    }
};
```

# 0049. 49. Group Anagrams

Difficulty: Medium Language: C++

## QUESTION

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

## EXAMPLE

No example extracted from the README.

## COMPLEXITY

Time:  $O(n \log n)$  Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> mp;
        for(string s: strs){
            string key = s;
            sort(key.begin(), key.end());
            mp[key].push_back(s);
        }
        vector<vector<string>> ans;
        for(auto x: mp){
            ans.push_back(x.second);
        }
        return ans;
    }
};
```

# 0053. 53. Maximum Subarray

Difficulty: Medium Language: C++

## QUESTION

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

## EXAMPLE

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]` Output: 6 Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int sum = 0;
        int maxsum = INT_MIN;
        for(int i = 0; i < nums.size(); i++){
            sum += nums[i];
            maxsum = max(maxsum, sum);
            if(sum < 0){
                sum = 0;
            }
        }
        return maxsum;
    }
};
```

# 0076. 76. Minimum Window Substring

Difficulty: Hard Language: C++

## QUESTION

Given two strings *s* and *t* of lengths *m* and *n* respectively, return the minimum window substring of *s* such that every character in *t* (including duplicates) is included in the window. If there is no such substring, return the empty string "".

## EXAMPLE

Input: *s* = "ADOBECODEBANC", *t* = "ABC" Output: "BANC" Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string *t*.

## COMPLEXITY

Time:  $O(n)$  average Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int>n,w;
        for(char c:t) n[c]++;
        int l = 0,start=0;
        int req = n.size();
        int have = 0;
        int len = INT_MAX ;
        for(int r = 0; r<s.size(); r++){
            w[s[r]]++;
            if(n.count(s[r]) && w[s[r]]==n[s[r]]){
                have++;
            }
            while(have == req){
                if(r-l+1 < len){
                    len = r-l+1;
                    start = l;
                }
                w[s[l]]--;
                if(n.count(s[l])&&w[s[l]]<n[s[l]]){
                    have--;
                }
            }
        }
        return s.substr(start, len);
    }
};
```



```
}  
l++;  
}  
}  
if(len == INT_MAX) return "";  
return s.substr(start,len);  
}  
};
```

# 0081. 81. Search in Rotated Sorted Array II

Difficulty: Medium Language: C++

## QUESTION

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

## EXAMPLE

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0` Output: `true`

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    bool search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target)
                return true;
            // Duplicates: cannot determine which half is sorted
            if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
                left++;
                right--;
            }
            // Left half is sorted
            else if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            // Right half is sorted
            else {

```

```
if (nums[mid] < target && target <= nums[right])  
    left = mid + 1;  
else  
    right = mid - 1;  
}  
}  
return false;  
}  
};
```

# 0094. 94. Binary Tree Inorder Traversal

Difficulty: Easy Language: C++

## QUESTION

Given the root of a binary tree, return the inorder traversal of its nodes' values.

## EXAMPLE

No example extracted from the README.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
 * right(right) {}
 * };
 */
class Solution {
public:
    vector<int> ans;
    vector<int> inorderTraversal(TreeNode* root) {
        if(root == NULL)
            return ans;
        inorderTraversal(root->left);
        ans.push_back(root->val);
        inorderTraversal(root->right);
        return ans;
    }
};
```

# 0098. 98. Validate Binary Search Tree

Difficulty: Medium Language: C++

## QUESTION

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

## EXAMPLE

Input: root = [2,1,3] Output: true

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
 * right(right) {}
 * };
 */
class Solution {
public:
    bool check(TreeNode* root, long long low, long long high){
        if(root == NULL) return true;
        if (root->val <= low || root->val >= high)
            return false;
        return check(root->left, low, root->val)&&check(root->right, root->val, high);
    }
    bool isValidBST(TreeNode* root) {
        return check(root, LLONG_MIN, LLONG_MAX);
    }
};
```

# 0100. 100. Same Tree

Difficulty: Easy Language: C++

## QUESTION

Given the roots of two binary trees *p* and *q*, write a function to check if they are the same or not.

## EXAMPLE

Input: *p* = [1,2,3], *q* = [1,2,3] Output: true

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
 * right(right) {}
 * };
 */
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if(p==NULL && q==NULL)return true;
        if(p==NULL || q==NULL)return false;
        if(p->val!=q->val) return false;
        return isSameTree(p->left,q->left)&&isSameTree(p->right,q->right);
    }
};
```

# 0101. 101. Symmetric Tree

Difficulty: Easy Language: C++

## QUESTION

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

## EXAMPLE

Input: root = [1,2,2,3,4,4,3] Output: true

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
 * right(right) {}
 * };
 */
class Solution {
public:
    bool mirror(TreeNode* left, TreeNode* right) {
        if (left == NULL && right == NULL)
            return true;
        if (left == NULL || right == NULL)
            return false;
        if (left->val != right->val)
            return false;
        return mirror(left->left, right->right) &&
            mirror(left->right, right->left);
    }
    bool isSymmetric(TreeNode* root) {
```

```
if (root == NULL)
return true;
return mirror(root->left, root->right);
}
};
```



# 0121. 121. Best Time to Buy and Sell Stock

Difficulty: Easy Language: C++

## QUESTION

You are given an array prices where prices[i] is the price of a given stock on the ith day.

## EXAMPLE

Input: prices = [7,1,5,3,6,4] Output: 5 Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minprice = prices[0];
        int profit = 0;
        for(int i =1;i<prices.size();i++){
            minprice= min(minprice,prices[i]);
            profit = max(profit,prices[i]-minprice);
        }
        return profit;
    }
};
```

# 0122. 122. Best Time to Buy and Sell Stock II

Difficulty: Medium Language: C++

## QUESTION

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

## EXAMPLE

Input: prices = [7,1,5,3,6,4] Output: 7 Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4. Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3. Total profit is 4 + 3 = 7.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit=0;
        for(int i = 1;i<prices.size();i++){
            if(prices[i]>prices[i-1]){
                profit = profit + prices[i] - prices[i-1];
            }
        }
        return profit;
    }
};
```

# 0128. 128. Longest Consecutive Sequence

Difficulty: Medium Language: C++

## QUESTION

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

## EXAMPLE

Input: `nums = [100,4,200,1,3,2]` Output: 4 Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

## COMPLEXITY

Time:  $O(n)$  average Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> st(nums.begin(),nums.end());
        int ans =0;
        for(int x:st){
            if(st.find(x-1)==st.end()){
                int current =x;
                int count =1;
                while(st.find(current+1)!=st.end()){
                    current++;
                    count++;
                }
                ans=max(ans,count);
            }
        }
        return ans;
    }
};
```

# 0135. 135. Candy

Difficulty: Hard Language: C++

## QUESTION

There are  $n$  children standing in a line.

## EXAMPLE

Input: ratings = [1,0,2] Output: 5 Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int candy(vector<int>& ratings) {
        int n = ratings.size();
        vector<int> candy(n, 1);
        for(int i = 1; i < n; i++) {
            if(ratings[i] > ratings[i-1])
                candy[i] = candy[i-1] + 1;
        }
        for(int i = n-2; i >= 0; i--) {
            if(ratings[i] > ratings[i+1])
                candy[i] = max(candy[i], candy[i+1] + 1);
        }
        int count = 0;
        for(int x : candy)
            count += x;
        return count;
    }
};
```

# 0189. 189. Rotate Array

Difficulty: Medium Language: C++

## QUESTION

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

## EXAMPLE

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3` Output: `[5,6,7,1,2,3,4]` Explanation: rotate 1 steps to the right: `[7,1,2,3,4,5,6]` rotate 2 steps to the right: `[6,7,1,2,3,4,5]` rotate 3 steps to the right: `[5,6,7,1,2,3,4]`

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k = k % nums.size();
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

# 0209. 209. Minimum Size Subarray Sum

Difficulty: Medium Language: C++

## QUESTION

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

## EXAMPLE

Input: `target = 7, nums = [2,3,1,2,4,3]` Output: 2 Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int l=0;
        int sum=0;
        int len = INT_MAX;
        for(int r=0;r<nums.size();++r){
            sum += nums[r];
            while(sum>=target){
                len = min(r-l+1,len);
                sum-=nums[l];
                ++l;
            }
        }
        if(len == INT_MAX) return 0;
        return len;
    }
};
```

# 0215. 215. Kth Largest Element in an Array

Difficulty: Medium Language: C++

## QUESTION

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

## EXAMPLE

Input: `nums = [3,2,1,5,6,4]`, `k = 2` Output: 5

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for(int x:nums){
            pq.push(x);
            if(pq.size()>k){
                pq.pop();
            }
        }
        return pq.top();
    }
};
```

# 0239. 239. Sliding Window Maximum

Difficulty: Hard Language: C++

## QUESTION

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

## EXAMPLE

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3` Output: `[3,3,5,5,6,7]` Explanation: Window position Max  
----- [1 3 -1] -3 5 3 6 7 3 1 [3 -1 -3] 5 3 6 7 3 1 3 [-1 -3 5] 3 6 7 5 1 3 -1 [-3 5 3] 6  
7 5 1 3 -1 -3 [5 3 6] 7 6 1 3 -1 -3 5 [3 6 7] 7

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> ans;
        for (int r = 0; r < nums.size(); r++) {
            // Remove elements outside the window
            while (!dq.empty() && dq.front() <= r - k) {
                dq.pop_front();
            }
            // Remove smaller elements
            while (!dq.empty() && nums[dq.back()] <= nums[r]) {
                dq.pop_back();
            }
            // Add current index
            dq.push_back(r);
            // Window size is k
            if (r >= k - 1) {
                ans.push_back(nums[dq.front()]);
            }
        }
    }
}
```



```
return ans;  
}  
};
```

# 0342. 342. Power of Four

Difficulty: Easy Language: C++

## QUESTION

Given an integer  $n$ , return true if it is a power of four. Otherwise, return false.

## EXAMPLE

Input:  $n = 16$  Output: true

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    bool isPowerOfFour(int n) {
        if(n<=0) return false;
        while(n%4==0){
            n/=4;
        }
        return n==1;
    }
};
```

# 0455. 455. Assign Cookies

Difficulty: Easy Language: C++

## QUESTION

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

## EXAMPLE

Input:  $g = [1,2,3]$ ,  $s = [1,1]$  Output: 1 Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.

## COMPLEXITY

Time:  $O(n \log n)$  Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int i = 0;
        int j = 0;
        int count = 0;
        while (i < g.size() && j < s.size()) {
            if (s[j] >= g[i]) {
                count++;
                i++;
                j++;
            }
            else {
                j++;
            }
        }
        return count;
    }
};
```

# 0628. 628. Maximum Product of Three Numbers

Difficulty: Easy Language: C++

## QUESTION

You are given an integer array `nums`.

## EXAMPLE

No example extracted from the README.

## COMPLEXITY

Time:  $O(n \log n)$  Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int maximumProduct(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        int a = nums[n-1] * nums[n-2] * nums[n-3];
        int b = nums[0] * nums[1] * nums[n-1];
        return max(a, b);
    }
};
```

# 0701. 784. Insert into a Binary Search Tree

Difficulty: Medium Language: C++

## QUESTION

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

## EXAMPLE

Input: root = [4,2,7,1,3], val = 5 Output: [4,2,7,1,3,5] Explanation: Another accepted tree is:

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
 * right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        // Empty position found
        if(root == NULL) {
            return new TreeNode(val);
        }
        // Go right
        if(val > root->val) {
            root->right = insertIntoBST(root->right, val);
        }
    }
};
```

```
}  
// Go left  
else {  
    root->left = insertIntoBST(root->left, val);  
}  
return root;  
}  
};
```

# 0896. 932. Monotonic Array

Difficulty: Easy Language: C++

## QUESTION

An array is monotonic if it is either monotone increasing or monotone decreasing.

## EXAMPLE

Input: nums = [1,2,2,3] Output: true

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
class Solution {
public:
    bool isMonotonic(vector<int>& nums) {
        bool increasing = true;
        bool decreasing = true;
        for(int i = 0; i < nums.size() - 1; i++) {
            if(nums[i] > nums[i+1])
                increasing = false;
            if(nums[i] < nums[i+1])
                decreasing = false;
        }
        return increasing || decreasing;
    }
};
```

# 1051. 1137. Height Checker

Difficulty: Easy Language: C++

## QUESTION

A school is trying to take an annual photo of all the students. The students are asked to stand in a single file line in non-decreasing order by height. Let this ordering be represented by the integer array `expected` where `expected[i]` is the expected height of the *i*th student in line.

## EXAMPLE

Input: `heights = [1,1,4,2,1,3]` Output: 3 Explanation: heights: [1,1,4,2,1,3] expected: [1,1,1,2,3,4]  
Indices 2, 4, and 5 do not match.

## COMPLEXITY

Time:  $O(n \log n)$  Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int heightChecker(vector<int>& heights) {
        int count = 0;
        vector<int> copy=heights;
        sort(copy.begin(),copy.end());
        for(int i=0;i<heights.size();i++){
            if(copy[i]!=heights[i]){
                count++;
            }
        }
        return count;
    }
};
```



# 1438. 1549. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

Difficulty: Medium Language: C++

## QUESTION

Given an array of integers `nums` and an integer `limit`, return the size of the longest non-empty subarray such that the absolute difference between any two elements of this subarray is less than or equal to `limit`.

## EXAMPLE

Input: `nums = [8,2,4,7]`, `limit = 4` Output: 2 Explanation: All subarrays are: `[8]` with maximum absolute diff  $|8-8| = 0 \leq 4$ . `[8,2]` with maximum absolute diff  $|8-2| = 6 > 4$ . `[8,2,4]` with maximum absolute diff  $|8-2| = 6 > 4$ . `[8,2,4,7]` with maximum absolute diff  $|8-2| = 6 > 4$ . `[2]` with maximum absolute diff  $|2-2| = 0 \leq 4$ . `[2,4]` with maximum absolute diff  $|2-4| = 2 \leq 4$ . `[2,4,7]` with maximum absolute diff  $|2-7| = 5 > 4$ . `[4]` with maximum absolute diff  $|4-4| = 0 \leq 4$ . `[4,7]` with maximum absolute diff  $|4-7| = 3 \leq 4$ . `[7]` with maximum absolute diff  $|7-7| = 0 \leq 4$ . Therefore, the size of the longest subarray is 2.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    int longestSubarray(vector<int>& nums, int limit) {
        deque<int> maxi;
        deque<int> mini;
        int l = 0;
        int ans = 0;
        for (int r = 0; r < nums.size(); r++) {
            // Maintain decreasing deque for maximum
            while (!maxi.empty() && nums[maxi.back()] < nums[r]) {
                maxi.pop_back();
            }
            maxi.push_back(r);
            // Maintain increasing deque for minimum
            while (!mini.empty() && nums[mini.back()] > nums[r]) {
                mini.pop_back();
            }
            mini.push_back(r);
```

```
// Window invalid
while (nums[maxi.front()] - nums[mini.front()] > limit) {
    if (maxi.front() == l)
        maxi.pop_front();
    if (mini.front() == l)
        mini.pop_front();
    l++;
}
ans = max(ans, r - l + 1);
}
return ans;
}
};
```

# 1980. 2107. Find Unique Binary String

Difficulty: Medium Language: C++

## QUESTION

Given an array of strings `nums` containing `n` unique binary strings each of length `n`, return a binary string of length `n` that does not appear in `nums`. If there are multiple answers, you may return any of them.

## EXAMPLE

Input: `nums = ["01","10"]` Output: `"11"` Explanation: `"11"` does not appear in `nums`. `"00"` would also be correct.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    string findDifferentBinaryString(vector<string>& nums) {
        string ans = "";
        for (int i = 0; i < nums.size(); i++) {
            if (nums[i][i] == '0')
                ans += '1';
            else
                ans += '0';
        }
        return ans;
    }
};
```

# 2426. 2513. Number of Pairs Satisfying Inequality

Difficulty: Hard Language: C++

## QUESTION

You are given two 0-indexed integer arrays `nums1` and `nums2`, each of size `n`, and an integer `diff`. Find the number of pairs  $(i, j)$  such that:

## EXAMPLE

Input: `nums1 = [3,2,5]`, `nums2 = [2,2,1]`, `diff = 1` Output: 3 Explanation: There are 3 pairs that satisfy the conditions: 1.  $i = 0, j = 1$ :  $3 - 2 \leq 2 - 2 + 1$ . Since  $i < j$  and  $1 \leq 1$ , this pair satisfies the conditions. 2.  $i = 0, j = 2$ :  $3 - 5 \leq 2 - 1 + 1$ . Since  $i < j$  and  $-2 \leq 2$ , this pair satisfies the conditions. 3.  $i = 1, j = 2$ :  $2 - 5 \leq 2 - 1 + 1$ . Since  $i < j$  and  $-3 \leq 2$ , this pair satisfies the conditions. Therefore, we return 3.

## COMPLEXITY

Time:  $O(n)$  Space:  $O(1)$

## SOLUTION

```
class Solution {
public:
    long long mergeSort(vector<long long>& a, int l, int r, int diff) {
        if(l >= r) return 0;
        int mid = l + (r - l) / 2;
        long long count = 0;
        count += mergeSort(a, l, mid, diff);
        count += mergeSort(a, mid + 1, r, diff);
        // Count pairs: i in left, j in right
        int j = mid + 1;
        for(int i = l; i <= mid; i++) {
            while(j <= r && a[i] > a[j] + diff)
                j++;
            count += r - j + 1;
        }
        // Normal merge
        vector<long long> temp;
        int i = l;
        j = mid + 1;
        while(i <= mid && j <= r) {
            if(a[i] <= a[j])
```

```

temp.push_back(a[i++]);
else
temp.push_back(a[j++]);
}
while(i <= mid)
temp.push_back(a[i++]);
while(j <= r)
temp.push_back(a[j++]);
for(int k = l; k <= r; k++)
a[k] = temp[k - l];
return count;
}
long long numberOfPairs(vector<int>& nums1,vector<int>& nums2,int diff) {
int n = nums1.size();
vector<long long> a(n);
for(int i = 0; i < n; i++)
a[i] = (long long)nums1[i] - nums2[i];
return mergeSort(a, 0, n - 1, diff);
}
};

```

# 3718. 4080. Smallest Missing Multiple of K

Difficulty: Easy Language: C++

## QUESTION

Given an integer array `nums` and an integer `k`, return the smallest positive multiple of `k` that is missing from `nums`.

## EXAMPLE

No example extracted from the README.

## COMPLEXITY

Time:  $O(n)$  average Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    int missingMultiple(vector<int>& nums, int k) {
        unordered_set<int> st;
        for(int x : nums){
            st.insert(x);
        }
        for(int i=1;i<=nums.size()+1;i++){
            int x = k*i;
            if(st.find(x)==st.end()){
                return x;
            }
        }
        return -1;
    }
};
```

# 3867. 4242. Sum of GCD of Formed Pairs

Difficulty: Medium Language: C++

## QUESTION

You are given an integer array `nums` of length `n`.

## EXAMPLE

No example extracted from the README.

## COMPLEXITY

Time:  $O(n \log n)$  Space:  $O(n)$

## SOLUTION

```
class Solution {
public:
    long long gcdSum(vector<int>& nums) {
        vector<int> g;
        int m = 0;
        int a = 0;
        for(int i = 0; i < nums.size(); i++){
            m = max(m, nums[i]);
            a = gcd(nums[i], m);
            g.push_back(a);
        }
        long long sum = 0;
        int b = 0;
        sort(g.begin(), g.end());
        for(int i = 0; i < g.size()/2; i++){
            b = gcd(g[i], g[g.size()-1-i]);
            sum += b;
        }
        return sum;
    }
};
```